

Name: Syeda Pakeeza Bukhari
SAP ID: 70174432

-> Github Link: [Churn Prediction Repository](#)

1. Data Loading and Understanding

Dataset file: WA_Fn-UseC_-Telco-Customer-Churn.csv

First 5 rows:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineS
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

Dataset shape: (7043, 21)

Column names:

['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'Intern

Data types:

0

customerID	object
gender	object
SeniorCitizen	int64
Partner	object
Dependents	object
tenure	int64
PhoneService	object
MultipleLines	object
InternetService	object
OnlineSecurity	object
OnlineBackup	object
DeviceProtection	object
TechSupport	object
StreamingTV	object
StreamingMovies	object
Contract	object
PaperlessBilling	object
PaymentMethod	object
MonthlyCharges	float64
TotalCharges	object
Churn	object

Basic statistics:

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
customerID	7043	7043	3186-AJIEK	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612	0.0	0.0	0.0	0.0	1.0
Partner	7043	2	No	3641	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Dependents	7043	2	No	4933	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481	0.0	9.0	29.0	55.0	72.0
PhoneService	7043	2	Yes	6361	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN	NaN	NaN	NaN	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN	NaN	NaN	NaN	NaN	NaN
DeviceProtection	7043	3	No	3095	NaN	NaN	NaN	NaN	NaN	NaN	NaN
TechSupport	7043	3	No	3473	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	7043	2	Yes	4171	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047	18.25	35.5	70.35	89.85	118.75
TotalCharges	7043	6531		11	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Churn	7043	2	No	5174	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Target variable: Churn

Target distribution:

count

Churn

No	5174
----	------

Yes 1869

dtype: int64

Target distribution (%):

proportion

Churn

No	73.46
----	-------

Yes 26.54

Important columns

- **customerID**: Unique identifier for a customer.
- **gender**: Customer's gender.
- **SeniorCitizen**: Indicates whether the customer is a senior citizen.
- **Partner**: Whether the customer has a partner.
- **Dependents**: Whether the customer has dependents.
- **tenure**: Number of months the customer has stayed with the company.
- **PhoneService / InternetService**: Services used by the customer.
- **Contract**: Customer's contract type.
- **PaymentMethod**: Method used for payment.
- **MonthlyCharges**: Customer's monthly charge.
- **TotalCharges**: Total amount charged to the customer.
- **Churn**: Target variable showing whether the customer left the company (Yes) or stayed (No).

2. Data Preprocessing

```
dtype: int64
Number of duplicate rows: 0

Unique values in TotalCharges:
['29.85', '1889.5', '108.15', '1840.75', '151.65', '820.5', '1949.4', '301.9', '3046.05', '3487.95']
```

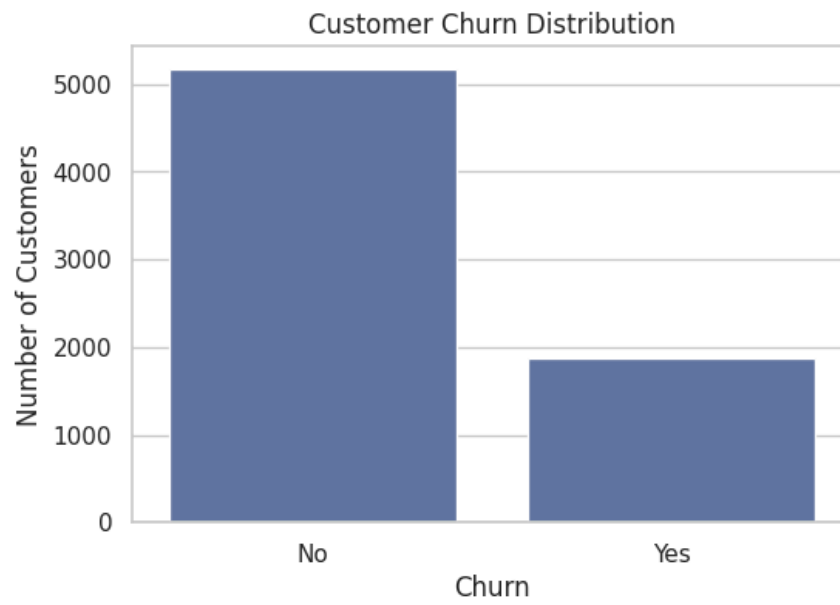
Missing values after converting TotalCharges:

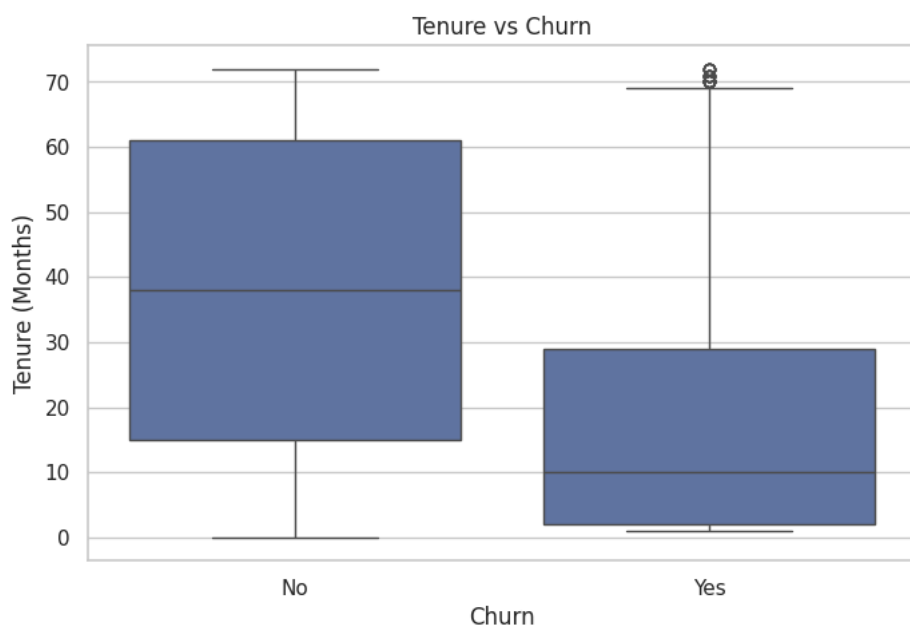
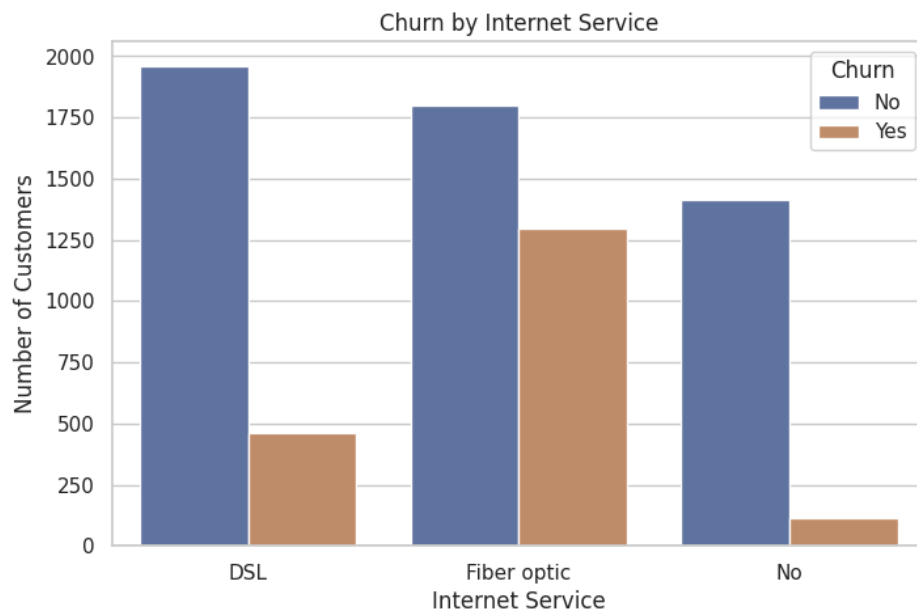
0

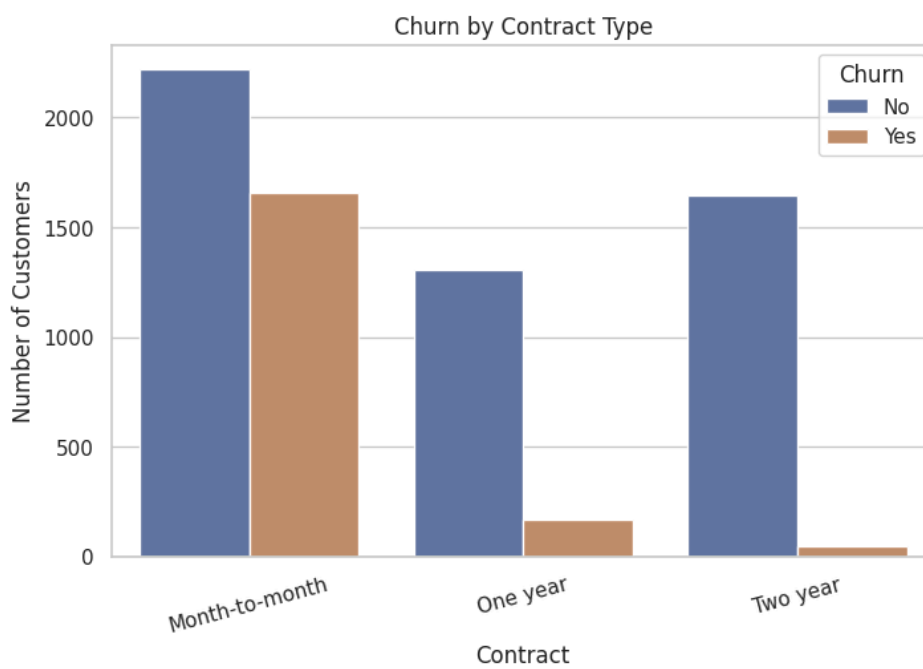
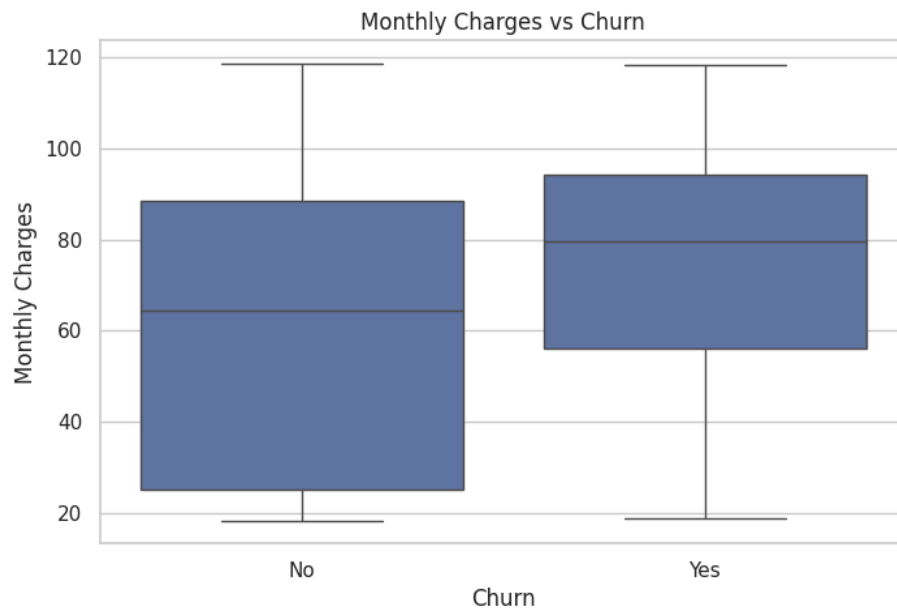
TotalCharges	11
--------------	----

```
dtype: int64
Duplicate rows removed: 0
Final dataset shape after basic cleaning: (7043, 21)
```

3. Exploratory Data Analysis







4. Feature Engineering and Encoding

```
X shape: (7043, 19)
y shape: (7043,)
```

Target mapping: No = 0, Yes = 1

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	No
0	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	No	No	No	No	Month-to-month	Yes	Electronic check	
1	Male	0	No	No	34	Yes	No	DSL	Yes	No	Yes	No	No	No	One year	No	Mailed check	
2	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	No	No	No	No	Month-to-month	Yes	Mailed check	
3	Male	0	No	No	45	No	No phone service	DSL	Yes	No	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	
4	Female	0	No	No	2	Yes	No	Fiber optic	No	No	No	No	No	No	Month-to-month	Yes	Electronic check	

Numerical features:

['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']

Categorical features:

['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod']

Preprocessing strategy

- Numerical columns are filled using the median and standardized.
- Categorical columns are filled using the most frequent value and converted into numerical form using One-Hot Encoding.
- customerID is removed because it is an identifier and does not provide useful predictive information.

5. Training and Testing

5. Training and Testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

Training samples: 5634
Testing samples: 1409

# Model 1: Logistic Regression
logistic_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])

# Model 2: Decision Tree
decision_tree_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(
        random_state=42,
        max_depth=5
    ))
])

logistic_model.fit(X_train, y_train)
decision_tree_model.fit(X_train, y_train)
print("Both models trained successfully.")
```

... Both models trained successfully.

6. Model Evaluation

==== Logistic Regression =====

Accuracy : 0.8055

Precision: 0.6572

Recall : 0.5588

F1-Score : 0.604

Classification Report:

	precision	recall	f1-score	support
No Churn	0.85	0.89	0.87	1035
Churn	0.66	0.56	0.60	374
accuracy			0.81	1409
macro avg	0.75	0.73	0.74	1409
weighted avg	0.80	0.81	0.80	1409

Confusion Matrix:

[[926 109]

[165 209]]

==== Decision Tree =====

Accuracy : 0.7984

Precision: 0.6347

Recall : 0.5668

F1-Score : 0.5989

Classification Report:

	precision	recall	f1-score	support
No Churn	0.85	0.88	0.87	1035
Churn	0.63	0.57	0.60	374
accuracy			0.80	1409
macro avg	0.74	0.72	0.73	1409
weighted avg	0.79	0.80	0.79	1409

Confusion Matrix:

[[913 122]

[162 212]]

```
# Compare the models
comparison = pd.DataFrame([logistic_results, tree_results])
display(comparison)

best_model_name = comparison.loc[comparison["F1-Score"].idxmax(), "Model"]

if best_model_name == "Logistic Regression":
    final_model = logistic_model
else:
    final_model = decision_tree_model

print("Selected final model based on F1-Score:", best_model_name)
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.805536	0.657233	0.558824	0.604046
1	Decision Tree	0.798439	0.634731	0.566845	0.598870

Selected final model based on F1-Score: Logistic Regression

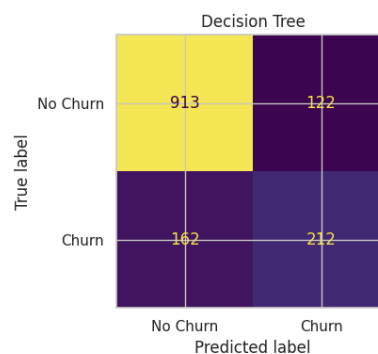
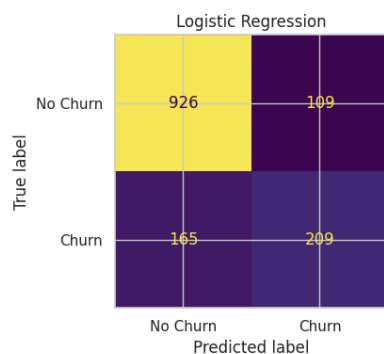
Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
# Confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

ConfusionMatrixDisplay(
    confusion_matrix=logistic_cm,
    display_labels=["No Churn", "Churn"]
).plot(ax=axes[0], values_format="d", colorbar=False)
axes[0].set_title("Logistic Regression")

ConfusionMatrixDisplay(
    confusion_matrix=tree_cm,
    display_labels=["No Churn", "Churn"]
).plot(ax=axes[1], values_format="d", colorbar=False)
axes[1].set_title("Decision Tree")

plt.tight_layout()
plt.show()
```



7. Application / Prediction

```
sample_customer = X_test.iloc[0].copy()

print("Sample customer information:")
display(sample_customer)

prediction = final_model.predict(sample_customer)[0]
prediction_label = "Churn" if prediction == 1 else "No Churn"
print("Predicted result:", prediction_label)
```

Sample customer information:

gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod
437	Male	0	Yes	Yes	72	Yes	Yes	Fiber optic	Yes	Yes	Yes	Yes	Yes	Two year	Yes	Credit card (automatic)

Predicted result: No Churn

```
sample_input = X.iloc[0].copy()

prediction = final_model.predict(sample_input)[0]
prediction_label = "Churn" if prediction == 1 else "No Churn"

print("Input customer:")
display(sample_input)
print("Model prediction:", prediction_label)
```

Input customer:

gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod
0	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	No	No	No	Month-to-month	Yes	Electronic check

Model prediction: Churn

8. Conclusion

In this project, two classification models, Logistic Regression and Decision Tree, were trained to predict customer churn. Logistic Regression achieved an accuracy of 80.55%, precision of 65.72%, recall of 55.88%, and F1-score of 60.40%. Decision Tree achieved an accuracy of 79.84%, precision of 63.47%, recall of 56.68%, and F1-score of 59.89%. Based on the F1-score used for model comparison, Logistic Regression was selected as the final model. The results show that the model can identify patterns related to customer churn, although some customers are still incorrectly classified. A telecom company could use such a system to identify customers who may be at risk of leaving and take appropriate retention actions.